

# **LAPORAN TUGAS 3**

**Mata Kuliah Sistem Operasi**



Di Susun Oleh :

Muhammad Rafi Amiruddin

Nim (2410651027)

**Jurusan Teknik Informatika**

**Fakultas Teknik**

**Universitas Muhammadiyah Jember**

**30 Oktober 2025**

## DAFTAR ISI

|                                   |   |
|-----------------------------------|---|
| LAPORAN TUGAS 3.....              | 1 |
| DAFTAR ISI .....                  | i |
| 1    ISI LAPORAN .....            | 1 |
| 1.1    Tugas 1.....               | 1 |
| 1.1.1    Kalkulator Paralel ..... | 1 |
| 1.2    Tugas 2.....               | 3 |
| 1.2.1    File Processing.....     | 3 |
| 1.3    Tugas 3.....               | 7 |
| 1.3.1    Producer-Consumer .....  | 7 |

# 1 ISI LAPORAN

## 1.1 Tugas 1

### 1.1.1 Kalkulator Paralel

- Buat program yang:
  1. Menerima input 2 angka dari user
  2. Menghitung penjumlahan, pengurangan, perkalian, dapat pembagian
  3. Setiap operasi dikerjakan oleh thread berbeda
  4. Tampilkan semua hasil setelah semua thread selesai

- Buat File **kalkulator\_paralel.c**

Yang didalam file berisi syntax :

```
#include <stdio.h>
#include <pthread.h>

// Struktur untuk menyimpan data input dan hasil
typedef struct {
    double a, b;
    double hasil;
} Data;

// Fungsi untuk tiap operasi
void* tambah(void* arg) {
    Data* data = (Data*)arg;
    data->hasil = data->a + data->b;
    pthread_exit(NULL);
}

void* kurang(void* arg) {
    Data* data = (Data*)arg;
    data->hasil = data->a - data->b;
    pthread_exit(NULL);
}

void* kali(void* arg) {
    Data* data = (Data*)arg;
    data->hasil = data->a * data->b;
    pthread_exit(NULL);
}

void* bagi(void* arg) {
    Data* data = (Data*)arg;
    if (data->b != 0)
```

```

        data->hasil = data->a / data->b;
    else
        data->hasil = 0; // hindari pembagian nol
    pthread_exit(NULL);
}

int main() {
    double x, y;
    pthread_t t1, t2, t3, t4;
    Data dTambah, dKurang, dKali, dBagi;

    printf("=== KALKULATOR PARALEL ===\n");
    printf("Masukkan angka pertama : ");
    scanf("%lf", &x);
    printf("Masukkan angka kedua : ");
    scanf("%lf", &y);

    // Set data untuk masing-masing thread
    dTambah.a = dKurang.a = dKali.a = dBagi.a = x;
    dTambah.b = dKurang.b = dKali.b = dBagi.b = y;

    // Buat thread untuk setiap operasi
    pthread_create(&t1, NULL, tambah, &dTambah);
    pthread_create(&t2, NULL, kurang, &dKurang);
    pthread_create(&t3, NULL, kali, &dKali);
    pthread_create(&t4, NULL, bagi, &dBagi);

    // Tunggu semua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);

    // Tampilkan hasil setelah semua selesai
    printf("\n=== HASIL PERHITUNGAN ===\n");
    printf("Penjumlahan : %.2lf\n", dTambah.hasil);
    printf("Pengurangan : %.2lf\n", dKurang.hasil);
    printf("Perkalian : %.2lf\n", dKali.hasil);
    printf("Pembagian : %.2lf\n", dBagi.hasil);

    return 0;
}

```

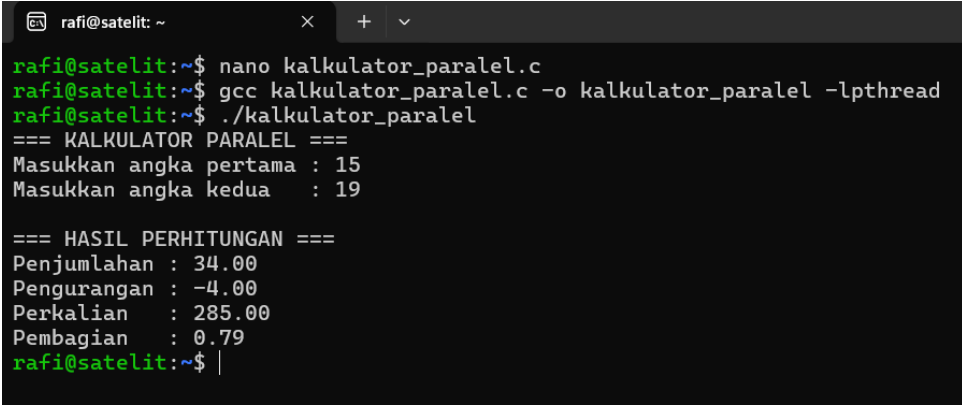
- Lalu Compile dan jalankan dengan `gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread` dan `./kalkulator_paralel`

Ini adalah Gambaran hasil kalkulator paralel

Program ini merupakan kalkulator paralel yang menggunakan multithreading (pthread) untuk menghitung empat operasi aritmetika dasar penjumlahan, pengurangan, perkalian, dan pembagian secara bersamaan (paralel).

1. pengguna memasukkan dua angka (misalnya 15 dan 19).
2. Program membuat empat thread, masing-masing untuk satu operasi matematika.
3. Setiap thread menghitung hasilnya sendiri tanpa menunggu yang lain selesai.
4. Setelah semua thread selesai, hasil ditampilkan bersamaan:
  - Penjumlahan: 34
  - Pengurangan: -4
  - Perkalian: 285
  - Pembagian: 0.79

Program ini menunjukkan cara pemrosesan paralel di C dengan pthread, di mana beberapa operasi dapat dilakukan secara simultan untuk meningkatkan efisiensi komputasi.



```

rafi@satelit: ~
rafi@satelit:~$ nano kalkulator_paralel.c
rafi@satelit:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -lpthread
rafi@satelit:~$ ./kalkulator_paralel
=== KALKULATOR PARALEL ===
Masukkan angka pertama : 15
Masukkan angka kedua  : 19

=== HASIL PERHITUNGAN ===
Penjumlahan : 34.00
Pengurangan : -4.00
Perkalian   : 285.00
Pembagian   : 0.79
rafi@satelit:~$
  
```

## 1.2 Tugas 2

### 1.2.1 File Processing

- Buat program yang :
  1. Membaca file teks
  2. Menghitung jumlah kata menggunakan 2 thread (bagi file jadi 2 bagian)
  3. Gabungkan hasilnya

#### 4. Bandingkan waktu eksekusi dengan versi single-thread

- Buat file **file\_processing.c**

**File tersebut di isi dengan syntax :**

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <time.h>

typedef struct {
    char *text;
    long start;
    long end;
    int count;
} ThreadData;

// Fungsi untuk menghitung kata di bagian tertentu
void* hitung_kata(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    int inWord = 0;
    int count = 0;
    for (long i = data->start; i < data->end; i++) {
        char c = data->text[i];
        if (c == ' ' || c == '\n' || c == '\t' || c == '\r' || c == '\0') {
            if (inWord) {
                count++;
                inWord = 0;
            }
        } else {
            inWord = 1;
        }
    }
    data->count = count;
    pthread_exit(NULL);
}

// Fungsi untuk membaca isi file
char* baca_file(const char* filename, long* ukuran) {
    FILE* file = fopen(filename, "r");
    if (!file) {
        printf("Gagal membuka file %s\n", filename);
        exit(1);
    }
    fseek(file, 0, SEEK_END);
    *ukuran = ftell(file);
    rewind(file);
```

```

char* buffer = (char*)malloc(*ukuran + 1);
fread(buffer, 1, *ukuran, file);
buffer[*ukuran] = '\0';
fclose(file);
return buffer;
}

// Hitung kata secara single-thread
int hitung_kata_single(char* text, long size) {
    int inWord = 0;
    int count = 0;
    for (long i = 0; i < size; i++) {
        char c = text[i];
        if (c == ' ' || c == '\n' || c == '\t' || c == '\r' || c == '\0') {
            if (inWord) {
                count++;
                inWord = 0;
            }
        } else {
            inWord = 1;
        }
    }
    return count;
}

int main() {
    char filename[100];
    printf("=== FILE PROCESSING (Word Count) ===\n");
    printf("Masukkan nama file teks: ");
    scanf("%s", filename);

    long size;
    char* text = baca_file(filename, &size);

    printf("\nUkuran file: %ld byte\n", size);

    // === SINGLE THREAD ===
    clock_t start_single = clock();
    int total_single = hitung_kata_single(text, size);
    clock_t end_single = clock();
    double waktu_single = (double)(end_single - start_single) /
CLOCKS_PER_SEC;

    // === MULTI THREAD (2 THREAD) ===
    pthread_t t1, t2;
    ThreadData d1, d2;

    long mid = size / 2;

```

```

d1.text = d2.text = text;
d1.start = 0;
d1.end = mid;
d2.start = mid;
d2.end = size;

clock_t start_multi = clock();

pthread_create(&t1, NULL, hitung_kata, &d1);
pthread_create(&t2, NULL, hitung_kata, &d2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);

int total_multi = d1.count + d2.count;
clock_t end_multi = clock();
double waktu_multi = (double)(end_multi - start_multi) /
CLOCKS_PER_SEC;

// === HASIL ===
printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Jumlah kata (Single Thread) : %d\n", total_single);
printf("Jumlah kata (Multi Thread) : %d\n", total_multi);
printf("\nWaktu eksekusi (Single Thread) : %.6f detik\n",
waktu_single);
printf("Waktu eksekusi (Multi Thread) : %.6f detik\n", waktu_multi);

free(text);
return 0;
}

```

- Lalu Compile dan Jalankan dengan gcc file\_processing.c -o file\_processing -lpthread dan ./file\_processing

Berikut adalah hasil dari pembuatan file processing

Program ini adalah program penghitungan jumlah kata (word count) dalam sebuah file teks menggunakan dua pendekatan:

1.

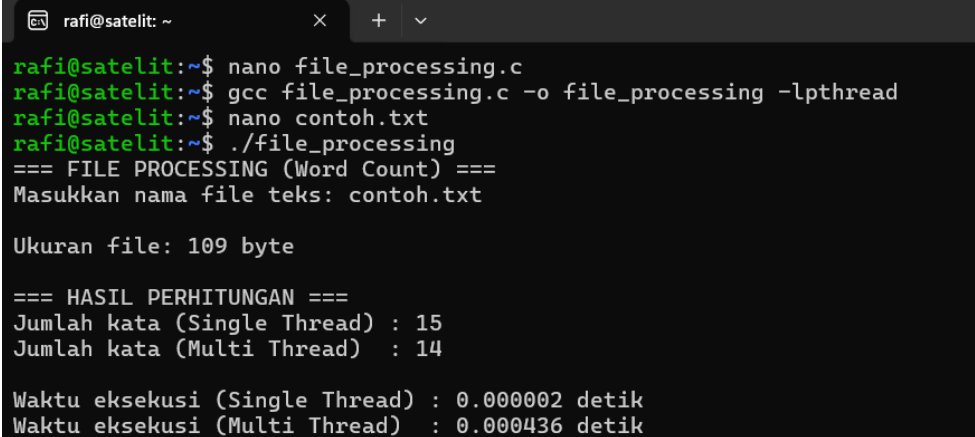
- Single Thread adalah menghitung kata secara berurutan (satu proses).
- Multi Thread adalah membagi file menjadi dua bagian dan menghitung jumlah kata secara paralel menggunakan pthread.

1. Program meminta nama file teks (contoh: contoh.txt)
2. File dibaca dan ukurannya ditampilkan (misal: 109 byte)
3. Dua metode dijalankan:



- **Single Thread:** menghitung jumlah kata secara linear (hasil: 15).
  - **Multi Thread:** menghitung kata dengan dua thread paralel (hasil: 14).
4. Program menampilkan waktu eksekusi masing-masing metode untuk membandingkan efisiensi.

Program ini menunjukkan perbandingan kinerja antara pemrosesan sekuensial dan paralel (multithreading) pada operasi file sederhana. Walaupun multithreading biasanya lebih cepat untuk file besar, pada file kecil justru bisa lebih lambat karena overhead pembuatan thread.



```

rafi@satelit: ~$ nano file_processing.c
rafi@satelit:~$ gcc file_processing.c -o file_processing -lpthread
rafi@satelit:~$ nano contoh.txt
rafi@satelit:~$ ./file_processing
=== FILE PROCESSING (Word Count) ===
Masukkan nama file teks: contoh.txt

Ukuran file: 109 byte

=== HASIL PERHITUNGAN ===
Jumlah kata (Single Thread) : 15
Jumlah kata (Multi Thread) : 14

Waktu eksekusi (Single Thread) : 0.000002 detik
Waktu eksekusi (Multi Thread) : 0.000436 detik

```

### 1.3 Tugas 3

#### 1.3.1 Producer-Consumer

- Buat simulasi producer-consumer dengan:
  1. Thread producer yang menghasilkan angka random
  2. Thread consumer yang memproses angka tersebut
  3. Gunakan mutex untuk sinkronisasi
  4. Buffer maksimal 10 item
- Buat File producer\_consumer.c

File tersebut berisi syntax :

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h> // untuk sleep()
#include <time.h>

#define BUFFER_SIZE 10

```

```

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer

pthread_mutex_t mutex;
pthread_cond_t notFull;
pthread_cond_t notEmpty;

// Fungsi untuk menambahkan item ke buffer
void produce(int item) {
    pthread_mutex_lock(&mutex);

    // Tunggu jika buffer penuh
    while (count == BUFFER_SIZE) {
        printf("[Producer] Buffer penuh, menunggu...\n");
        pthread_cond_wait(&notFull, &mutex);
    }

    buffer[count] = item;
    count++;
    printf("[Producer] Menghasilkan item: %d (buffer terisi %d/%d)\n",
item, count, BUFFER_SIZE);

    pthread_cond_signal(&notEmpty); // beri sinyal ke consumer
    pthread_mutex_unlock(&mutex);
}

// Fungsi untuk mengambil item dari buffer
int consume() {
    pthread_mutex_lock(&mutex);

    // Tunggu jika buffer kosong
    while (count == 0) {
        printf("[Consumer] Buffer kosong, menunggu...\n");
        pthread_cond_wait(&notEmpty, &mutex);
    }

    int item = buffer[count - 1];
    count--;
    printf("[Consumer] Mengonsumsi item: %d (buffer terisi %d/%d)\n",
item, count, BUFFER_SIZE);

    pthread_cond_signal(&notFull); // beri sinyal ke producer
    pthread_mutex_unlock(&mutex);

    return item;
}

// Thread producer
void* producer(void* arg) {
    while (1) {

```

```

        int item = rand() % 100; // angka random 0–99
        produce(item);
        sleep(1); // simulasi waktu produksi
    }
}

// Thread consumer
void* consumer(void* arg) {
    while (1) {
        int item = consume();
        // simulasi waktu proses item
        printf("  -> [Consumer] Memproses item %d...\n", item);
        sleep(2);
    }
}

int main() {
    pthread_t prodThread, consThread;
    srand(time(NULL));

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&notFull, NULL);
    pthread_cond_init(&notEmpty, NULL);

    printf("=== SIMULASI PRODUCER-CONSUMER ===\n");

    // Buat thread producer dan consumer
    pthread_create(&prodThread, NULL, producer, NULL);
    pthread_create(&consThread, NULL, consumer, NULL);

    // Jalankan terus (CTRL + C untuk berhenti)
    pthread_join(prodThread, NULL);
    pthread_join(consThread, NULL);

    // Bersihkan resource
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&notFull);
    pthread_cond_destroy(&notEmpty);

    return 0;
}

```

- Lalu Compile dan jalankan dengan gcc producer\_consumer.c -o producer\_consumer -lpthread dan ./producer\_consumer

Program ini merupakan simulasi konsep Producer-Consumer menggunakan multithreading (pthread) dan sinkronisasi dengan mutex dan mungkin semaphore. Tujuannya adalah untuk menunjukkan bagaimana dua jenis

thread — *Producer* dan *Consumer* — bekerja sama menggunakan buffer bersama (shared buffer) secara aman tanpa konflik data.

1. Producer menghasilkan item (angka acak) dan menambahkannya ke buffer.
  - Jika buffer penuh (10/10), producer harus *menunggu* sampai ada ruang kosong.
2. Consumer mengambil item dari buffer dan “memprosesnya”.
  - Jika buffer kosong, consumer menunggu sampai ada item baru dari producer.
3. Program menampilkan status buffer setiap kali item ditambahkan atau dikonsumsi.

rogram ini menggambarkan konsep sinkronisasi dan komunikasi antar thread dalam pemrograman paralel, di mana resource bersama (buffer) harus dikelola secara hati-hati untuk menghindari race condition. Dengan menggunakan mutex dan kondisi (wait/signal), program memastikan producer dan consumer bekerja bergantian secara terkoordinasi sesuai kapasitas buffer.

```

rafi@satelit: ~
x + v
rafi@satelit:~$ nano producer_consumer.c
rafi@satelit:~$ gcc producer_consumer.c -o producer_consumer -lpthread
rafi@satelit:~$ ./producer_consumer
=== SIMULASI PRODUCER-CONSUMER ===
[Producer] Menghasilkan item: 31 (buffer terisi 1/10)
[Consumer] Mengonsumsi item: 31 (buffer terisi 0/10)
-> [Consumer] Memproses item 31...
[Producer] Menghasilkan item: 59 (buffer terisi 1/10)
[Producer] Menghasilkan item: 87 (buffer terisi 2/10)
[Consumer] Mengonsumsi item: 87 (buffer terisi 1/10)
-> [Consumer] Memproses item 87...
[Producer] Menghasilkan item: 57 (buffer terisi 2/10)
[Producer] Menghasilkan item: 5 (buffer terisi 3/10)
[Consumer] Mengonsumsi item: 5 (buffer terisi 2/10)
-> [Consumer] Memproses item 5...
[Producer] Menghasilkan item: 2 (buffer terisi 3/10)
[Consumer] Mengonsumsi item: 2 (buffer terisi 2/10)
-> [Consumer] Memproses item 2...
[Producer] Menghasilkan item: 73 (buffer terisi 3/10)
[Producer] Menghasilkan item: 18 (buffer terisi 4/10)
[Consumer] Mengonsumsi item: 18 (buffer terisi 3/10)
-> [Consumer] Memproses item 18...
[Producer] Menghasilkan item: 99 (buffer terisi 4/10)
[Producer] Menghasilkan item: 84 (buffer terisi 5/10)
[Consumer] Mengonsumsi item: 84 (buffer terisi 4/10)
-> [Consumer] Memproses item 84...
[Producer] Menghasilkan item: 61 (buffer terisi 5/10)
[Producer] Menghasilkan item: 81 (buffer terisi 6/10)
[Consumer] Mengonsumsi item: 81 (buffer terisi 5/10)
-> [Consumer] Memproses item 81...
[Producer] Menghasilkan item: 1 (buffer terisi 6/10)
[Producer] Menghasilkan item: 32 (buffer terisi 7/10)
[Consumer] Mengonsumsi item: 32 (buffer terisi 6/10)
-> [Consumer] Memproses item 32...
[Producer] Menghasilkan item: 42 (buffer terisi 7/10)
[Producer] Menghasilkan item: 70 (buffer terisi 8/10)
[Consumer] Mengonsumsi item: 70 (buffer terisi 7/10)
-> [Consumer] Memproses item 70...
[Producer] Menghasilkan item: 66 (buffer terisi 8/10)
[Producer] Menghasilkan item: 47 (buffer terisi 9/10)
[Consumer] Mengonsumsi item: 47 (buffer terisi 8/10)

```

```

rafi@satelit: ~
[Producer] Menghasilkan item: 46 (buffer terisi 9/10)
[Producer] Menghasilkan item: 83 (buffer terisi 10/10)
[Consumer] Mengonsumsi item: 83 (buffer terisi 9/10)
-> [Consumer] Memproses item 83...
[Producer] Menghasilkan item: 35 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 35 (buffer terisi 9/10)
-> [Consumer] Memproses item 35...
[Producer] Menghasilkan item: 51 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 51 (buffer terisi 9/10)
-> [Consumer] Memproses item 51...
[Producer] Menghasilkan item: 34 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 34 (buffer terisi 9/10)
-> [Consumer] Memproses item 34...
[Producer] Menghasilkan item: 82 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 82 (buffer terisi 9/10)
-> [Consumer] Memproses item 82...
[Producer] Menghasilkan item: 64 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 64 (buffer terisi 9/10)
-> [Consumer] Memproses item 64...
[Producer] Menghasilkan item: 2 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 2 (buffer terisi 9/10)
-> [Consumer] Memproses item 2...
[Producer] Menghasilkan item: 33 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 33 (buffer terisi 9/10)
-> [Consumer] Memproses item 33...
[Producer] Menghasilkan item: 82 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 82 (buffer terisi 9/10)
-> [Consumer] Memproses item 82...
[Producer] Menghasilkan item: 58 (buffer terisi 10/10)
[Producer] Buffer penuh, menunggu...
[Consumer] Mengonsumsi item: 58 (buffer terisi 9/10)
-> [Consumer] Memproses item 58...
[Producer] Menghasilkan item: 80 (buffer terisi 10/10)

```